

TRIALMATE AI

AI-Powered Clinical Trial Site Copilot

Developed By:

Abhishek Maskara

M.Sc. Medical Biotechnology

Advanced Diploma in Clinical SAS Programming

PROJECT JOURNEY

As someone pursuing a career in Clinical Research, Clinical SAS Programming, and eventually advanced clinical data sciences, I wanted to build a solution that addresses a practical challenge faced at clinical trial sites.

During my learning journey, I observed that newly onboarded Clinical Research Coordinators (CRCs) often struggle with understanding protocols, clinical research terminology, eligibility screening requirements, and investigator communication. These activities require significant training before a CRC becomes fully productive.

With the Agilisium AI-Assisted Development Challenge, I saw an opportunity to explore how Generative AI could be used to simplify these workflows and support CRC onboarding.

This led to the development of TrialMate AI.

PROJECT OBJECTIVE

The objective of TrialMate AI is to provide an AI-powered assistant that helps Clinical Research Coordinators perform common site-level activities through a simple and accessible interface.

The platform focuses on four key workflows:

- Clinical Research Knowledge Support
 - Protocol Interpretation
 - Patient Eligibility Screening
 - Investigator Brief Generation
-

SOLUTION OVERVIEW

TrialMate AI was developed using Python, Streamlit, and Google's Gemini API.

The application allows users to:

1. Ask clinical research questions and receive CRC-focused explanations.
 2. Paste protocol sections and automatically generate:
 - Plain Language Summaries
 - Inclusion Criteria
 - Exclusion Criteria
 - CRC Action Points
 3. Screen participants using predefined eligibility criteria.
 4. Generate investigator-ready briefing notes containing eligibility assessments and recommended next steps.
-

MY AI-ASSISTED DEVELOPMENT JOURNEY

One of the most valuable aspects of this project was learning how AI-assisted development works in practice.

This project was not simply about generating code.

It involved:

- Breaking down a problem into smaller components.
- Understanding application architecture.
- Writing and refining prompts.
- Debugging environment issues.
- Testing AI model behavior.
- Iteratively improving outputs.

Throughout development, I continuously refined prompts to ensure that Gemini responded as a Clinical Research Coordinator onboarding assistant rather than as a generic chatbot.

This significantly improved the relevance and quality of responses generated by the application.

DEVELOPMENT PROCESS AND ENGINEERING NOTES

During the development of TrialMate AI, I documented the complete engineering process from environment setup to deployment.

The project began with establishing the development environment using Python, Streamlit, and Google's Gemini API. Before writing application code, I validated Python installations, package management tools, and API connectivity.

One of the most valuable lessons came from debugging.

Initially, Streamlit repeatedly displayed its default demo application rather than the TrialMate AI interface. Instead of focusing on "why the code was not working," I learned to isolate and validate each component individually.

The debugging process included:

- Verifying Python file extensions and directory structures.
- Checking terminal execution paths.
- Testing Python execution independently.
- Testing Streamlit deployment independently.
- Using debugging statements such as:
 - `raise Exception("TRIALMATE TEST")`
 - `print("I reached here")`
 - Validating Streamlit port configurations.

The final solution involved deploying the application using an alternative port configuration.

Another important learning involved Gemini API integration.

To ensure reliable model access, I created dedicated testing scripts to:

- Validate API authentication.
- Enumerate available Gemini models.
- Test response generation independently from the main application.
- Troubleshoot model compatibility issues.

Prompt engineering became one of the most important aspects of the project.

The initial responses generated by Gemini were generic and not sufficiently focused on clinical research workflows. Through multiple iterations, I refined prompts to ensure that TrialMate AI consistently behaved as a Clinical Research Coordinator onboarding assistant.

Separate prompts were designed and optimized for:

- Clinical Research Knowledge Assistant
- Protocol Interpreter
- Investigator Brief Generator

This iterative refinement significantly improved the relevance, structure, and usefulness of generated outputs.

The experience demonstrated that successful AI applications require a combination of software engineering, prompt engineering, debugging methodology, and user-centered workflow design.

TECHNICAL CHALLENGES AND DEBUGGING

The project involved several real-world technical challenges.

Environment Setup

The initial setup required validation of Python installations, package management, Streamlit configuration, and Gemini API integration.

Streamlit Deployment Issue

One of the major issues encountered was that Streamlit repeatedly displayed its default demo page instead of the TrialMate AI application.

To resolve this issue, I performed systematic debugging including:

- File validation
- Directory verification
- Python execution testing
- Streamlit execution testing
- Port troubleshooting

This experience helped me understand the importance of isolating and validating individual components of an application.

Gemini API Integration

While integrating Gemini, I encountered model compatibility issues and API quota limitations.

I learned how to:

- Test model availability
- Validate API responses
- Troubleshoot quota-related errors
- Select appropriate deployment models

Prompt Engineering

Perhaps the most important learning was prompt engineering.

I discovered that the quality of AI responses depends heavily on the instructions provided to the model.

Through multiple iterations, I improved prompts to ensure:

- Clinical relevance
- Structured responses
- CRC-focused explanations
- Professional formatting
- Reduced hallucinations

KEY LEARNINGS

Through this project, I gained practical experience in:

- Python Development
- Streamlit Application Development
- Gemini API Integration
- Prompt Engineering
- Debugging Methodologies
- User-Centered Design
- Rapid Prototyping
- AI-Assisted Development

The project demonstrated that successful AI applications require both technical implementation and careful workflow design.

FUTURE ENHANCEMENTS

Future versions of TrialMate AI may include:

- Protocol PDF Upload
 - Dynamic Eligibility Extraction
 - Multi-Study Support
 - EDC Integration
 - Investigator Dashboard
 - Expanded Clinical Research Knowledge Base
-

CONCLUSION

TrialMate AI represents my first end-to-end AI-assisted software development project focused on clinical research workflows.

The project allowed me to combine my background in Medical Biotechnology, Clinical Research, and Clinical SAS with modern Generative AI technologies.

Beyond the final software product, the most valuable outcome was the hands-on experience gained in debugging, prompt engineering, workflow design, and AI-assisted development.

This project strengthened my understanding of how Generative AI can be applied to solve real-world challenges within the clinical research ecosystem.